

Introduction to Profiling

Ye Liu

July 5, 2025

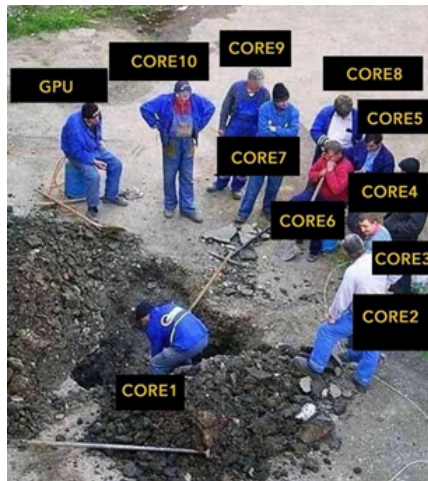
Zhejiang University Supercomputing Team

1. Introduction
2. CPU Profile
3. GPU Profile

Introduction

Why Performance Profiling Matters

- **Performance Bottlenecks** are often not where you think
- **Optimization without data** leads to wasted effort
- **Scalability Issues** appear with larger data sets or systems

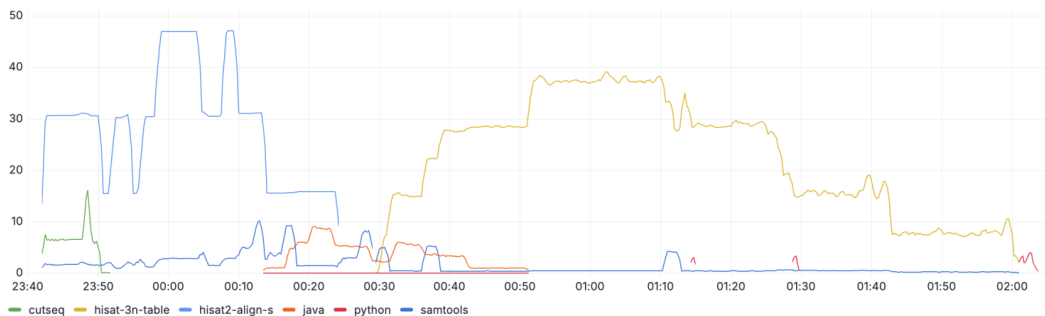


Real World Example

m5C-RNA:

Graph

Lines Bars Points Stacked lines Stacked bars

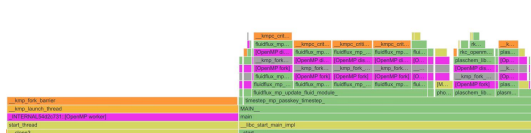


Real World Example

- Profiling of a HPC application

Elapsed Time[?]: 300.427s >

⌵ CPU Time ⓘ:	5488.014s
Effective Time ⓘ:	1001.315s
⌵ Spin Time ⓘ:	2635.347s 🚩
Imbalance or Serial Spinning ⓘ:	2441.683s 🚩
Lock Contention ⓘ:	82.775s
Other ⓘ:	110.890s
⌶ Overhead Time ⓘ:	1851.353s 🚩
Total Thread Count:	32
Paused Time ⓘ:	0s



Without Profiling

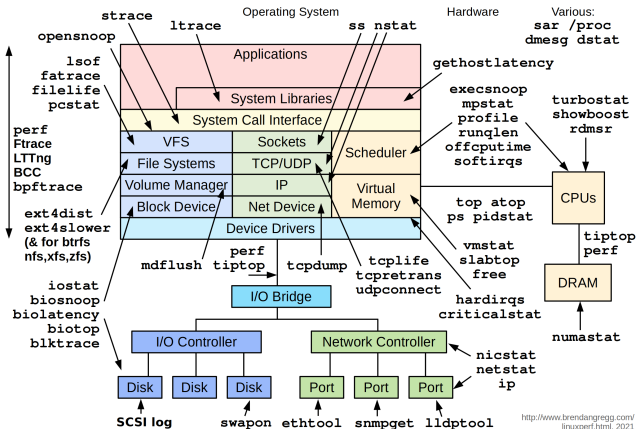
- Just optimize “obvious” bottlenecks
- Spend time on wrong code sections
- Limited understanding of system behavior

With Profiling

- Data-driven optimization
- Focus on actual hotspots
- Understand memory patterns
- Measure optimization impact

Profiling Categories

Linux Performance Observability Tools



<http://www.brendangregg.com/linuxperf.html>, 2021

Everything can be profiled!

- **System:** Overall system performance, focus on memory bandwidth, peak float performance, etc.
- **Program:** Focus on specific program behavior, e.g., memory access patterns, branch prediction, etc.

Why do we need both?

- System profiling gives us details about the hardware and the optimization upper bound(or how to build a more powerful system)
- Program profiling makes us able to touch the machine's limits

System Profiler Recommendation

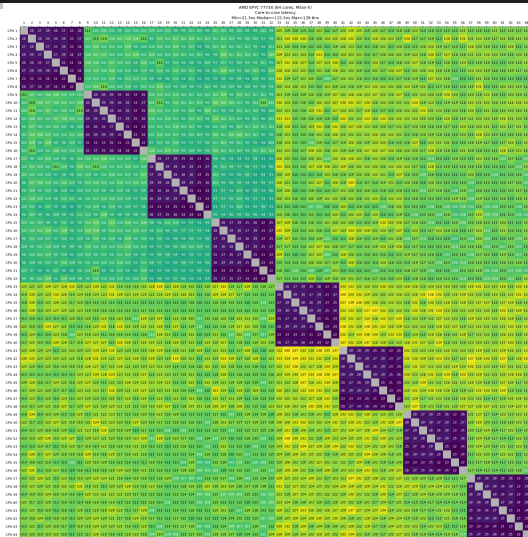
- **CPUFP**: A tool for measuring the peak performance of kinds of SIMD instructions
- <https://github.com/pigirons/cpufp>

```
liuye@M700:~/cpufp$ ./cpufp --thread_pool=[0-7]
Number Threads: 8
Thread Pool Binding: 0 1 2 3 4 5 6 7
```

Instruction Set	Core Computation	Peak Performance
AMX_INT8	MM(s32,s8,s8)	41.876 TOPS
AMX_INT8	MM(s32,s8,u8)	41.858 TOPS
AMX_INT8	MM(s32,u8,s8)	41.805 TOPS
AMX_INT8	MM(s32,u8,u8)	41.866 TOPS
AMX_BF16	MM(f32,bf16,bf16)	20.935 TFLOPS
AVX512_VNNI	DP4A(s32,u8,s8)	5.6459 TOPS
AVX512_VNNI	DP2A(s32,s16,s16)	2.8233 TOPS
AVX_VNNI	DP4A(s32,u8,s8)	2.8224 TOPS
AVX_VNNI	DP2A(s32,s16,s16)	1.4115 TOPS
AVX512_BF16	DP2A(f32,bf16,bf16)	617.27 GFLOPS
AVX512_FP16	FMA(f16,f16,f16)	2.8218 TFLOPS
AVX512F	FMA(f32,f32,f32)	1.3086 TFLOPS
AVX512F	FMA(f64,f64,f64)	649.94 GFLOPS
FMA	FMA(f32,f32,f32)	664.73 GFLOPS
FMA	FMA(f64,f64,f64)	352.73 GFLOPS
AVX	ADD(MUL(f32,f32),f32)	488.57 GFLOPS
AVX	ADD(MUL(f64,f64),f64)	244.31 GFLOPS
SSE	ADD(MUL(f32,f32),f32)	263.45 GFLOPS
SSE2	ADD(MUL(f64,f64),f64)	131.52 GFLOPS

System Profiler Recommendation

- **core-to-core-latency:** A tool for measuring the latency of core communication
- <https://github.com/nviennot/core-to-core-latency>



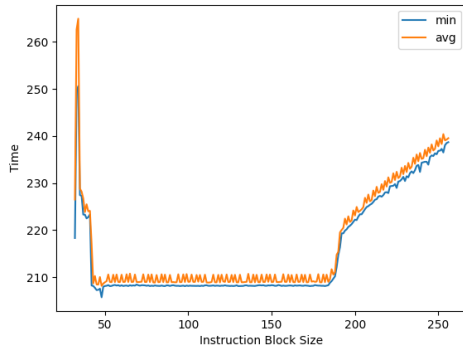
System Profiler Recommendation

- **STREAM:** A tool for measuring the memory bandwidth
- <https://github.com/jeffhammond/STREAM>

```
liuye@M700:~/STREAM$ ./stream_c.exe
-----
STREAM version $Revision: 5.10 $
-----
This system uses 8 bytes per array element.
-----
Array size = 10000000 (elements), Offset = 0 (elements)
Memory per array = 76.3 MiB (= 0.1 GiB).
Total memory required = 228.9 MiB (= 0.2 GiB).
Each kernel will be executed 10 times.
The *best* time for each kernel (excluding the first iteration)
will be used to compute the reported bandwidth.
-----
Number of Threads requested = 96
Number of Threads counted = 96
-----
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 4782 microseconds.
(= 4782 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-----
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-----
Function      Best Rate MB/s  Avg time     Min time     Max time
Copy:         226949.2    0.000785    0.000705    0.001192
Scale:        303522.7    0.000589    0.000527    0.000835
Add:          230087.5    0.001113    0.001043    0.001424
Triad:        267295.0    0.000911    0.000898    0.000955
-----
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-----
```

System Profiler Recommendation

- **cpu-micro-benchmark**: A tool for reverse engineering CPU microarchitecture
- <https://github.com/jiegec/cpu-micro-benchmarks>
- <https://jia.je/hardware/2024/12/27/cpu-uarch-reverse-engineering-method>



CPU Profile

- **Perf:** Linux kernel profiling tool, can be widely used(e.g. PAC)
- **Vtune:** Intel's comprehensive performance analysis tool

Key Features:

- CPU sampling and tracing
- Hardware counter access
- Call graph generation
- Memory profiling
- Low overhead

Common Commands:

- `perf top`
- `perf record`
- `perf report`
- `perf stat`

Reference: <https://jia.je/software/2024/12/10/linux-perf-pmu/>

Advantages:

- Rich GUI interface
- Advanced hotspot analysis
- Threading analysis
- Memory and cache analysis
- Intel hardware optimization

Analysis Types:

- Hotspots
- Threading
- MPI
- Memory Access
- Microarchitecture

Let's take a closer look.

GPU Profile

- **Nsight Systems:** System-wide performance analysis
- **Nsight Compute:** Detailed kernel analysis

Key Features:

- System-wide timeline view
- CPU and GPU activity correlation
- Memory transfer analysis
- Multi-GPU support
- Low overhead profiling

Use Cases:

- Application bottlenecks
- GPU utilization
- Data transfer optimization
- Multi-threading analysis

Capabilities:

- Detailed kernel metrics
- Memory throughput analysis
- Warp efficiency metrics
- Instruction analysis
- Performance limiters identification

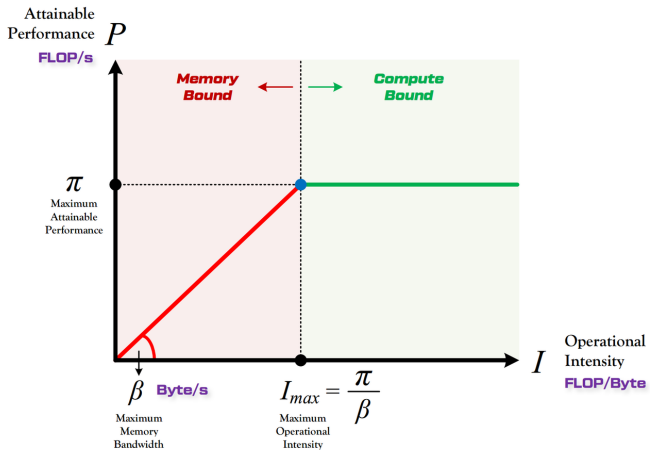
Analysis Focus:

- Kernel optimization
- Memory access patterns
- Occupancy analysis
- Compute throughput

Let's have a closer look.

- The computing platform: **computational capability** (FLOPS) and **memory bandwidth**.
- The algorithm or model: **total computations** and **memory accesses**.
- *"A model with A computations and B memory accesses, running on a system with computational capability C and memory bandwidth D , what is the theoretical upper bound of achievable performance E ?"*

Roofline Model



Thank You

Any Questions?